

# **Coursework Summary**

Shobanapriyan Chandrasegaran  
240248516  
MSC Applied Artificial Intelligence

## **1. INTRODUCTION**

This document summarizes the key steps, analytical decisions, and conclusions derived from the Jupyter notebook focused on exploratory data analysis (EDA), preprocessing, and modeling of the given epitope dataset.

The primary objective of the analysis was to prepare the data for a downstream modeling task aimed at developing a generalized model capable of predicting epitopes from embedding features of unseen protein sequences from a human pathogen. The analysis addressed data quality issues, reduced dimensionality, and ensured robustness to support the subsequent classification task.

## **2. EXPLORATORY DATA ANALYSIS (EDA)**

The initial step in the workflow involves loading and inspecting the dataset to understand its overall structure. EDA is a critical phase because it informs the choice of pre-processing techniques and provides early insights into potential issues such as missing values, data duplicates, outliers, and feature relevance.

### **2.1 Dataset Overview**

The dataset comprises 45,000 records with 1,284 columns, including metadata (Info\_organism\_id, Info\_pos, Info\_cluster), 1,280 numerical features, and the target (Class). All features are of type float64, ensuring data type consistency. Info\_cluster as metadata, cluster the features into 265 unique clusters.

### **2.2 Statistical Summary**

A descriptive statistical analysis is carried out for all features. Key statistics like mean, median, standard deviation, minimum, and maximum values provide insights into the spread and distribution of each feature. This step also helps identify skewness, potential transformation needs, and collinearity between features. Visualizations such as boxplots and histograms are used to enhance the interpretability of this summary.

### **2.3 Missing Values**

The presence of missing values in the dataset is visualized using the missingno library, which effectively highlights gaps in the data. Missing values can distort model performance and must be treated appropriately. In this analysis, the decision was made to apply a Simple Imputation

with the mean strategy. Missing values are imputed using the mean of the respective features, ensuring the central tendency is preserved without introducing bias.

## 2.4 Outlier Detection and Treatment

Outliers are another important consideration, especially in numerical data, where extreme values can skew results. This analysis utilizes the Interquartile Range (IQR) method to detect and flag outliers. Features that fall outside 1.5 times the IQR below the first quartile or above the third quartile are considered outliers. Once detected, these values are handled using simple imputation with the mean strategy to maintain dataset consistency and prevent these anomalies from adversely affecting the model. To ensure the outlier treatment is achieved successfully, the Principal Component Analysis (PCA) is applied, and the projection of two Principal Components (PCs) of the feature space is obtained, which is used to generate the box plot to analyze the outliers. Based on the box plot, the conclusion was that there are no outliers in the new feature space obtained from the PCs.

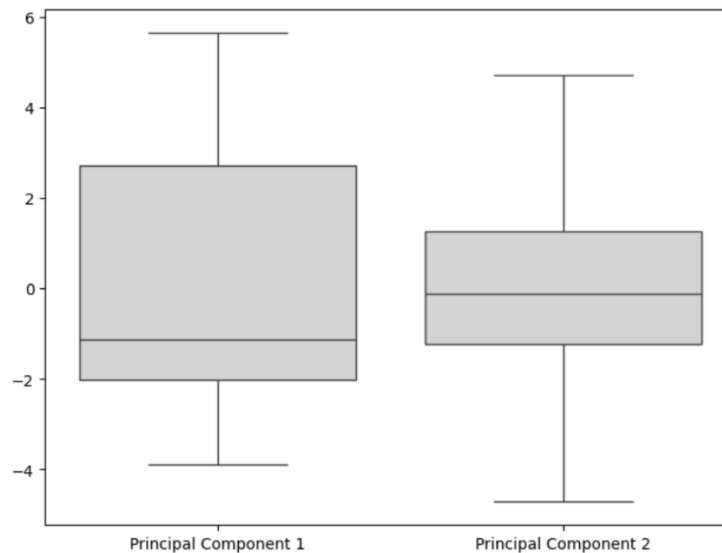


Figure 1: Obtained box plot on the obtain principal components

## 3. DATA PRE-PROCESSING

Based on the insights gained from EDA, several pre-processing techniques are applied to improve data quality and reduce redundancy. These include normalizing feature scales, dimensionality reduction, and addressing class imbalance.

### 3.1 Scaling and Normalization

Before applying dimensionality reduction, the dataset is standardized using **StandardScaler**, which transforms features to have a mean of zero and a standard deviation of one. Standardization is particularly important when using algorithms like PCA, which are sensitive to the scale of the data.

### 3.2 Class Imbalance Handling

The target variable (Class) is binary, with values -1 and 1, indicating an imbalanced classification problem (1 => 36668, -1 => 526). The majority class (1) dominates the dataset, which may bias model performance if unaddressed. SMOTE (Synthetic Minority Over-sampling Technique) is used to handle this class imbalance. This is often a crucial step, especially in classification problems where one class dominates. Techniques like SMOTE generate synthetic samples of the minority class, helping balance the class distribution and avoid bias in the model.

## 4. FEATURE REDUCTION

To address the potential curse of dimensionality and multicollinearity among features, Principal Component Analysis (PCA) is employed. PCA reduces the number of features by projecting them onto a new feature space that captures the maximum variance in the data. The selected optimal number of principle components (4) are obtained by fine tuning for a selected model (Random Forest). The cumulative variance obtained from the selected number of components is 0.3.

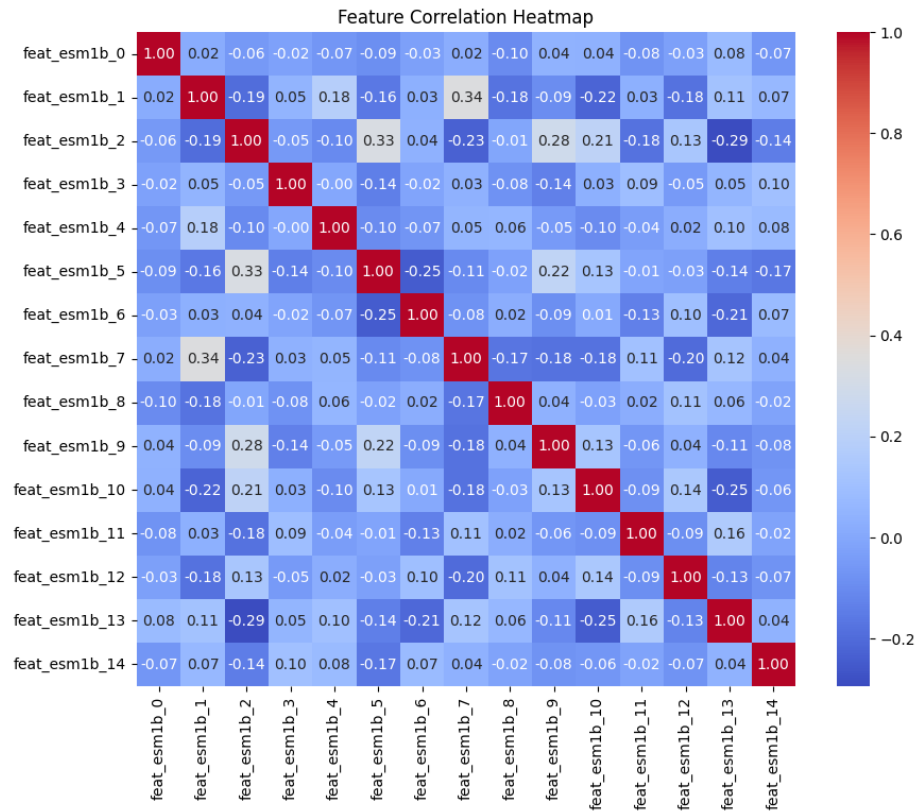


Figure 2: Feature Correlation Heatmap on selected features

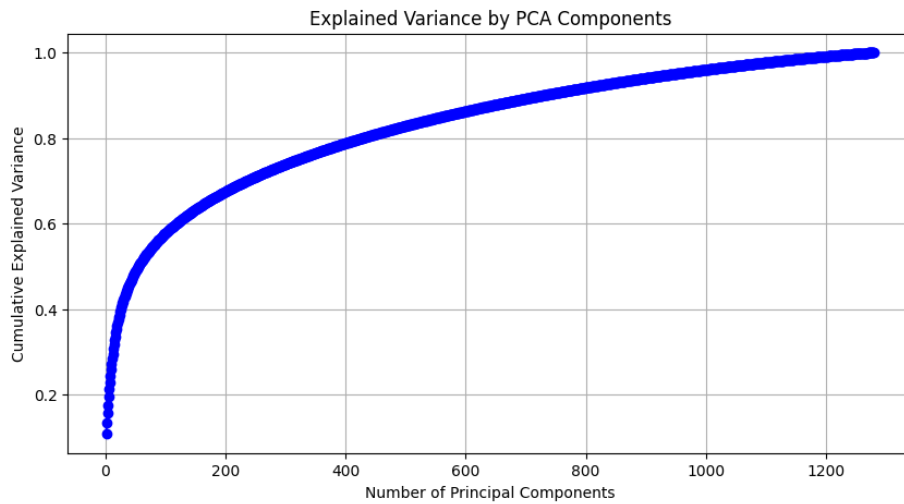


Figure 3: Explained Variance plot

## 5. MODELING AND ASSESSMENT

Machine learning classifiers includes **Logistic Regression (Linear classification model)**, and **Random Forest (Bagging model)** are used in modeling, and **balanced accuracy score** is used as validation metrics to compare the model performance since the given dataset is imbalanced. Additionally, confusion matrix, and classification report are also included in the notebook.

Logistic regression model is trained on the scaled train set with 1280 features is considered as baseline model, and the obtain balanced accuracy score 0.5034. Random forest model is selected and gone through multiple optimization cycle in the following order.

1. Trained on the scaled train set with 1280 features
2. Trained on the PCAs (40) of the scaled feature train set
3. Trained on the scaled and balanced train set with 1280 features using SMOTE
4. Trained on the PCAs (6) of the scaled and balanced train set– optimal PCAs number obtain by hyper parameter tuning

| Model   | Balanced Accuracy Score |
|---------|-------------------------|
| Model 1 | 0.5                     |
| Model 2 | 0.5                     |
| Model 3 | 0.4967                  |
| Model 4 | 0.5286                  |

Table 1: Results comparison with different models

High balanced accuracy obtained with model 4 trained on 6 PCAs of the scaled and balanced train set which is 0.5286.

## 5. CONCLUSIONS AND DISCUSSION

From the analysis, the following key conclusions are drawn:

- The dataset contains a high number of features, all numerical, with several missing values and some clear outliers.
- Simple imputation provides a quick and effective way to handle missing values without introducing artificial patterns into the data.
- The IQR method effectively identifies outliers, which are then treated to maintain the statistical consistency of the dataset.
- Standardization is a crucial pre-step for PCA and improves its performance by aligning feature scales.
- PCA successfully reduces the dimensionality of the data while preserving variance, thus simplifying the dataset and improving model interpretability.
- SMOTE successfully handled the data imbalance issue and improved the model accuracy.
- Combination of PCA + SMOTE further improve the model generalization and balanced accuracy which outperform the base line model.

Future steps would likely include different classifiers (boosting models including XGBoost, Adaboost, Catboost) on model training, apply different feature scaling methods (Robust scaling minmax scaling), efficient imputation methods, advanced model-based features selection approaches (MRMR) and different data imbalance handling (Eg:Adasyn) approaches.

## REFERENCES

- Kumar, A. (2023, November 24). *PCA explained variance concepts with Python example*. Vitalflux. Retrieved April 26, 2025, from <https://vitalflux.com/pca-explained-variance-concept-python-example/>
- Lemaître, G., Nogueira, F., & Aridas, C. K. (n.d.). *imblearn.over\_sampling.SMOTE* — imbalanced-learn 0.12.2 documentation. Retrieved April 26, 2025, from [https://imbalancedlearn.org/stable/references/generated/imblearn.over\\_sampling.SMOTE.html](https://imbalancedlearn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html)
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (n.d.). *sklearn.ensemble.RandomForestClassifier* — scikit-learn 1.4.2 documentation. Retrieved April 26, 2025, from <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (n.d.). *sklearn.linear\_model.LogisticRegression* — scikit-learn 1.4.2 documentation. Retrieved April 26, 2025, from [https://scikit-learn.org/stable/modules/generated/sklearn.linear\\_model.LogisticRegression.html](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html)
- Prabhu, P. (2023, February 20). *Outlier detection and removal using the IQR method*. Medium. <https://medium.com/@pp1222001/outlier-detection-and-removal-using-the-iqr-method-6fab2954315d>